

Настройка NGINX



Становимся суперпользователем

```
sudo su
```

Для начала обновляем список доступных пакетов

```
apt update
```

Устанавливаем nginx

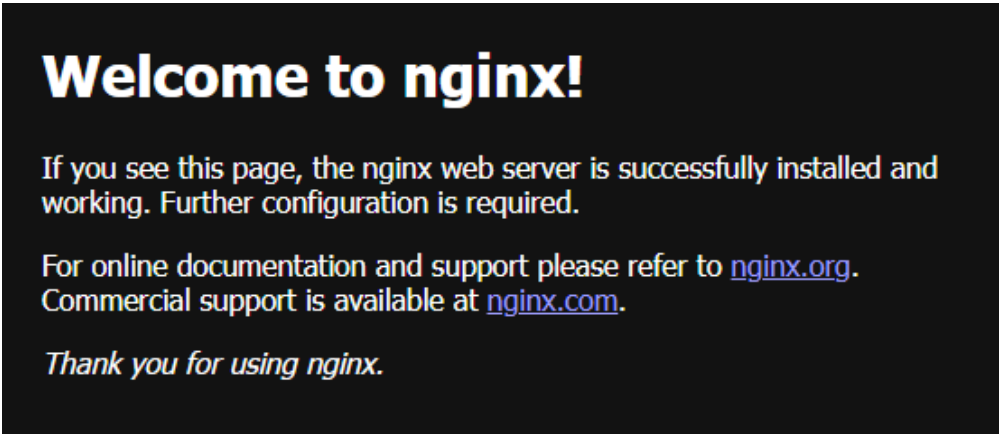
```
apt install -y nginx
```

Проверяем nginx

```
nginx -v → nginx version: nginx/1.24.0 (Ubuntu)
```

Если перейти в браузере по адресу вашего сервера, должно вернуться следующее

порт 80



Теперь изменим конфигурацию nginx для работы нашего приложения

Для начала очистим стандартный файл конфигурации

```
echo "" > /etc/nginx/sites-available/default
```

Теперь откроем этот файл и добавим конфигурацию

```
nano /etc/nginx/sites-available/default
```

```
upstream backend_servers {
    keepalive 60;
    keepalive_requests 100;
    server 127.0.0.1:8000 max_fails=3 fail_timeout=2s;
}

server {
    client_max_body_size 300M;

    location / {
        proxy_set_header Host $http_host;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_pass http://backend_servers;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Keep-Alive "timeout=60";
    }
}
```

Должно быть

Проверяем конфигурацию

```
nginx -t
```

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Перезагружаем nginx

```
systemctl restart nginx
```

Данная конфигурация Nginx состоит из двух основных блоков: определения группы серверов `upstream` и настройки прокси-сервера для обработки запросов.

1. Блок `upstream`

- `upstream backend_servers`: Определяет пул серверов с именем `backend_servers`, к которым будет отправляться трафик из блока `proxy_pass`.
- `keepalive 60`: Указывает, что Nginx будет поддерживать до 60 соединений к каждому серверу из пула одновременно в режиме `keep-alive`, что снижает накладные расходы на повторное создание соединений.
- `keepalive_requests 100`: Указывает максимальное количество запросов, которые могут быть выполнены по одному `keep-alive` соединению, прежде чем оно будет закрыто.
- `server 127.0.0.1:8000`: Указывает, что сервер в пуле расположен на `localhost` и слушает порт 8000.
- `max_fails=3`: Если сервер трижды подряд не ответит, он будет считаться недоступным.
- `fail_timeout=2s`: После признания сервера недоступным, попытки отправить запросы к нему возобновятся только через 2 секунды.

2. Блок `server`

`client_max_body_size 300M`: Ограничивает максимальный размер тела запроса до 300 МБ. Полезно для загрузки крупных файлов.

2.1. Блок `location`

Этот блок обрабатывает все запросы, начинающиеся с `/`.

- `proxy_set_header`: Настраивает заголовки, которые Nginx отправляет на серверы `backend_servers`.
 - `Host $http_host`: Передает хост из исходного HTTP-запроса (обычно это доменное имя).
 - `Upgrade $http_upgrade` и `Connection "upgrade"`: Используются для поддержки протоколов WebSocket или других обновляемых соединений.
 - `X-Real-IP $remote_addr`: Передает реальный IP-адрес клиента.
 - `X-Forwarded-For $proxy_add_x_forwarded_for`: Добавляет IP-адрес клиента в цепочку прокси-заголовков `X-Forwarded-For`.
 - `X-Forwarded-Proto $scheme`: Указывает, используется ли протокол HTTP или HTTPS.
- `proxy_pass http://backend_servers`: Указывает, что запросы должны перенаправляться на пул серверов, определенных в `upstream backend_servers`.
- `proxy_http_version 1.1`: Устанавливает версию протокола HTTP для прокси-соединений. HTTP/1.1 необходим для `keep-alive` и работы WebSocket.
- `proxy_set_header Connection ""`: Удаляет заголовок `Connection`, чтобы избежать конфликтов с управлением соединениями.
- `proxy_set_header Keep-Alive "timeout=60"`: Указывает, что сервер backend может держать соединение открытым в течение 60 секунд для `keep-alive`.

